Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Disentangled Accent Modeling for Neural Voice Conversion

Project Preview Presentations

Focus: Machine Learning & Neural Speech Processing

Team: Yupeng ZHAO • Weirong YE • An QIU

Research Background & Objectives

Context:

Voice Conversion (VC) aims to change a source speaker's identity to a target while preserving linguistic content.

Applications ：

Key use cases include movie dubbing, language learning through cross-lingual conversion, and speaking assistance.

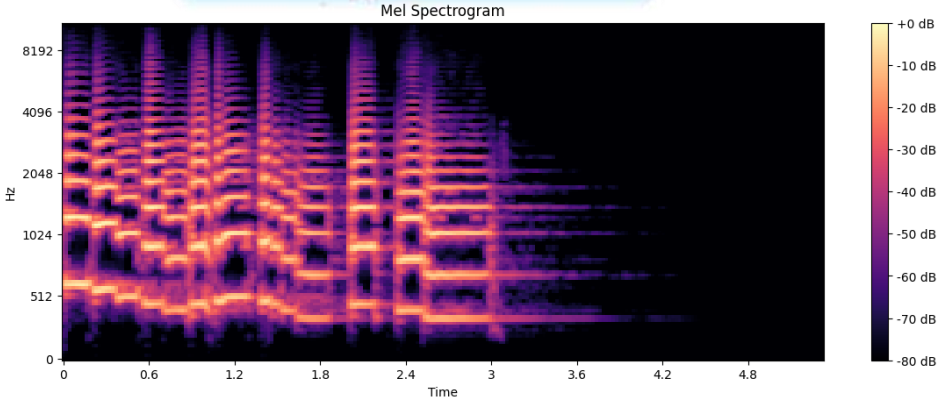
Problem Shooting:

Most high-quality methods require "parallel" utterances (the same text spoken by different people), which are rarely available in real-world scenarios.

Objective:

To develop a many-to-many VC framework that works without text labels or parallel data while achieving natural-sounding results.

** Dialects of English in UK and Ireland*



Feature Extraction: Mel-spectrogram Representation

Literature Review

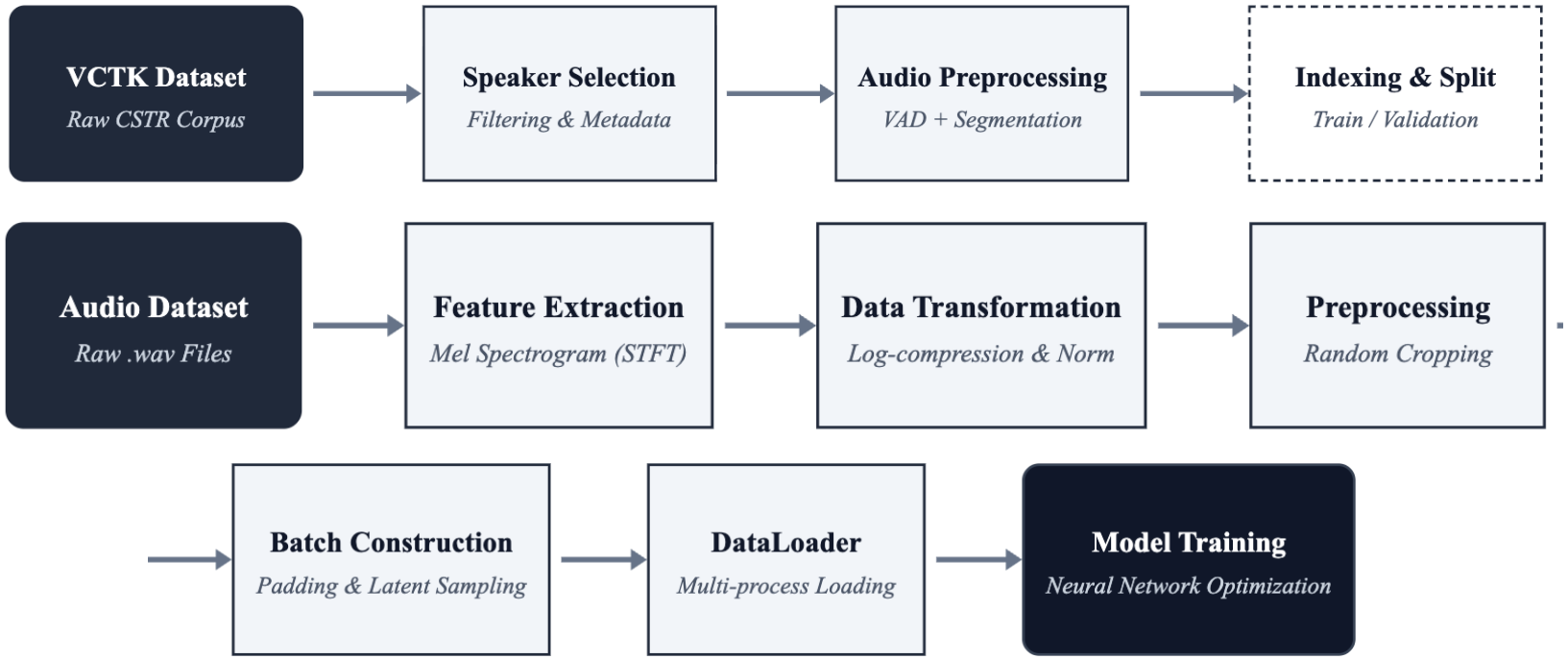
Data Base: VCTK Corpus (University of Edinburgh)

Used for neural voice conversion training

- **Scale: 110 speakers, ~400 utterances** per speaker (~**44 hours** total speech)
- **Speaker diversity:** Multi-speaker corpus with distinct speaker identities
- **Accent diversity:** Diverse regional English accents
- **Recording quality:** Studio-quality recordings in controlled acoustic environment

Source: University of Edinburgh VCTK Corpus (2019)

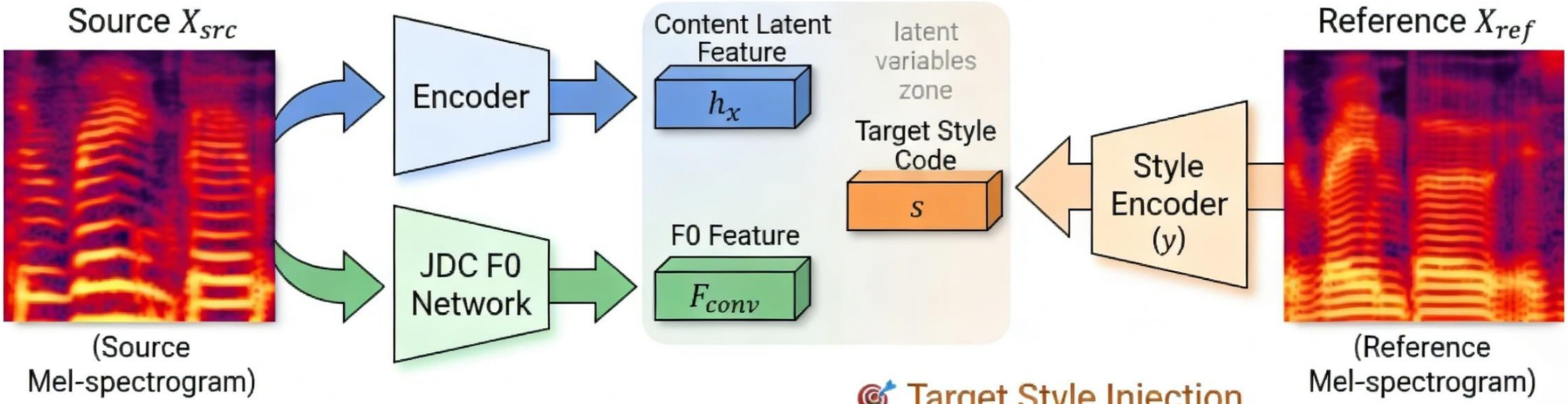
FIGURE 1: VCTK AUDIO PREPROCESSING PIPELINE (LANDSCAPE VIEW)



System Structure: Encoder

Source Audio Processing

Content Extraction: Source Mel-spectrogram X_{src} → Encoder → Content-latent features h_x (Speaker-independent)



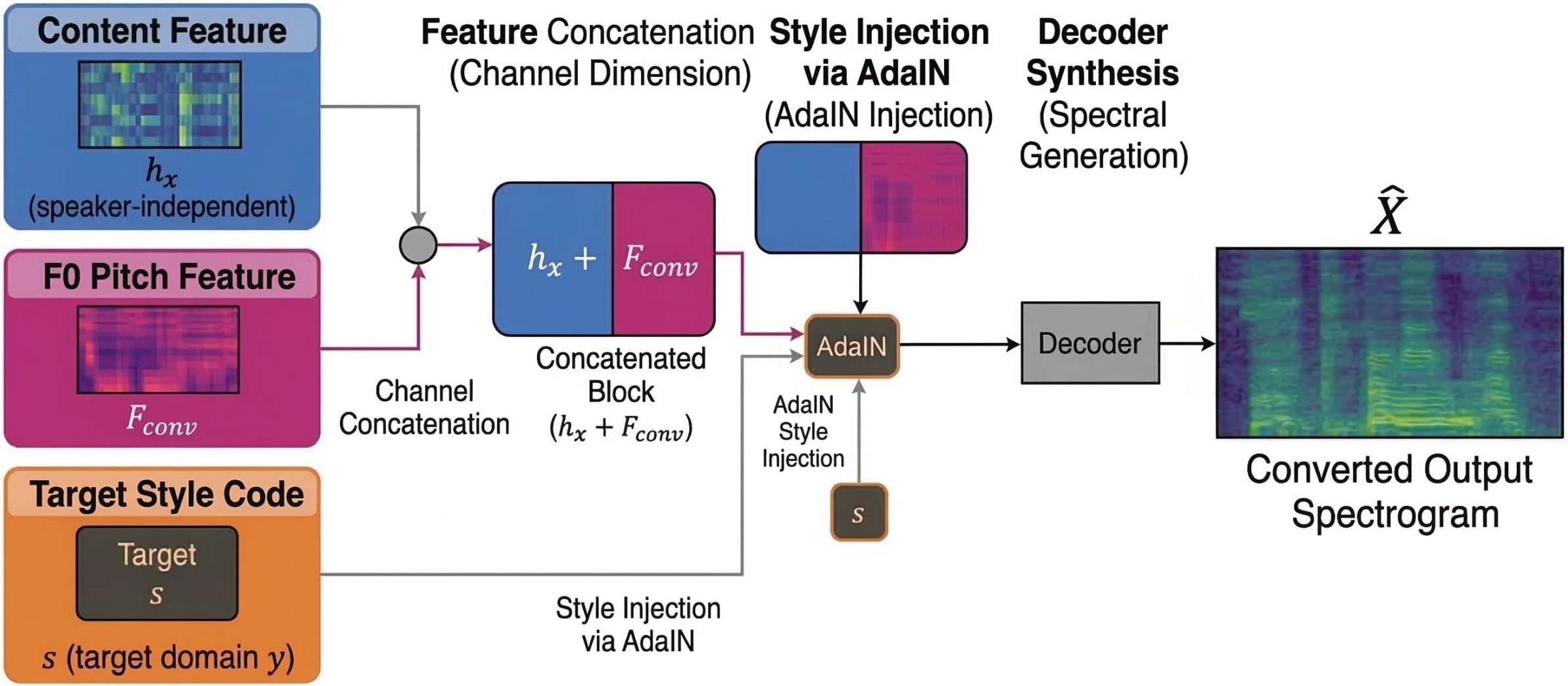
Pitch Preservation: Pre-trained JDC F0 Network extracts fundamental frequency feature F_{conv}

Target Style Injection

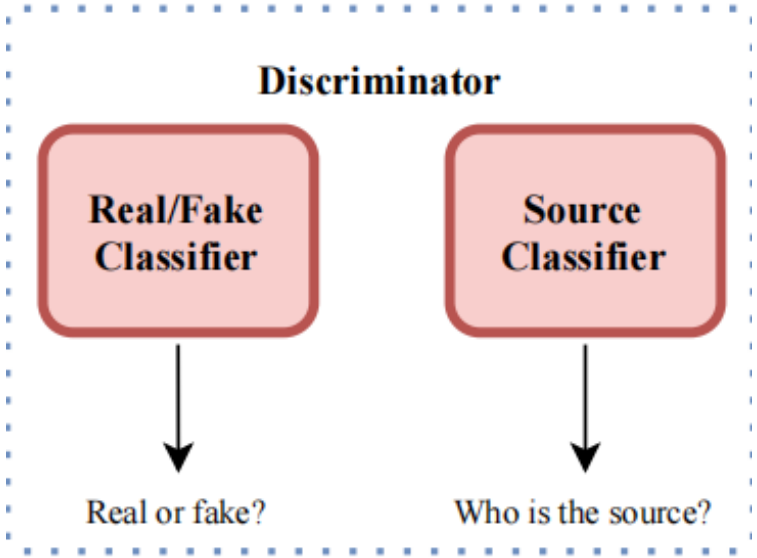
Style Encoding: Reference audio X_{ref} → Style Encoder → Domain-specific target style code s (Reference input required)

*(Note: Optional Mapping Network can generate random styles for diversity.)

System Framework - Decoder



System Framework: Discriminator



-Real/Fake Classifier → Whether the generated $\hat{X}$ looks like real speech from the target domain

-Source Classifier →The original source domain from which the generated sample came

X: *source mel-spectrogram*
G(X,s): *converted mel-spectrogram generated by the generator*
ysrc: *source speaker label*
ytrg: *target speaker label*
D(·,y): *output of the Real/Fake classifier in domain y*
C(·): *output of the Source classifier*
CE(·): *cross-entropy loss*

Core loss functions

1. Real/Fake Classifier Loss L_{adv}

$$L_{adv} = \mathbb{E}_{X,y_{src}}[\log D(X, y_{src})] + \mathbb{E}_{X,y_{trg},s}[\log(1 - D(G(X,s), y_{trg}))]$$

2. Source classifier loss L_{cls}

$$L_{cls} = \mathbb{E}_{X,y_{src},s}[CE(C(G(X,s)), y_{src})]$$

VOICE CONVERSION LOSS FUNCTIONS

Generative Adversarial Network (GAN)

- **Discriminator matching:** Feeds real/fake speech to the discriminator (targets 1/0). Uses **BCE** for binary classification.
- **Adversarial loss:** Feeds fake samples to the discriminator (target 1) using **BCE**. Penalizes less realistic generated sounds.

Style Conversion

- **Style loss:** Computes the difference between output and target style vectors. Uses **L1 loss** (simple, direct, and robust to large outliers).

Content & Prosody Preservation

- **Cycle loss:** Preserves core content by measuring the difference between original input and cycle-reconstruction via **L1 loss**.
- **ASR loss:** Constrains converted speech to preserve content by comparing high-level ASR features via **smooth L1 loss**.
- **F0 loss:** Preserves pitch and prosodic contour by comparing relative F0 contours via **L1 loss**.

Train.py

Algorithm 1 Overall Training Procedure

Load configuration and initialize logging tools
Load training and validation data
Build dataloaders
Load pretrained ASR and F0 networks
Construct StarGANv2-VC model and EMA model
Set up optimizer and scheduler
Initialize trainer
if a checkpoint exists **then**
    Restore model parameters
end if
for each training epoch **do**
    Run training and validation
    Record metrics and save checkpoints
end for

```python
# load pretrained ASR model
ASR_config = config.get('ASR_config', False)
ASR_path = config.get('ASR_path', False)
with open(ASR_config) as f:
        ASR_config = yaml.safe_load(f)
ASR_model_config = ASR_config['model_params']
ASR_model = ASRCNN(**ASR_model_config)
params = torch.load(ASR_path, map_location='cpu')['model']
ASR_model.load_state_dict(params)
_ = ASR_model.eval()

# load pretrained F0 model
F0_path = config.get('F0_path', False)
F0_model = JDCNet(num_class=1, seq_len=192)
params = torch.load(F0_path, map_location='cpu')['net']
F0_model.load_state_dict(params)
```